

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.12.17 星期三 11-12 节

实验题目：添加新的系统调用（进程信息）

一、实验内容

基于 lab8 的 schello，扩展系统调用功能，实现内核态遍历所有进程，输出每个进程的名字、PID、父进程 ID、进程状态，并附加学号和姓名，编写用户态测试程序，验证新系统调用的功能。

二、实现步骤

1、环境准备

依旧仅做增量编译。

运行 Linux6.16.12 内核并进入内核源码目录

```
LZH2311451@LZH2311451-pc:~$ uname -a
Linux LZH2311451-pc 6.16.12LZH2311451 #3 SMP PREEMPT_DYNAMIC Thu Jan  1 08:41:41
CST 2026 x86_64 x86_64 x86_64 GNU/Linux
LZH2311451@LZH2311451-pc:~$ cd /usr/src/linux
LZH2311451@LZH2311451-pc:/usr/src/linux$
```

2、添加系统调用

（1）声明系统调用原型（lab8 已完成，跳过）

vim include/linux/syscalls.h

```
asmlinkage long sys_schello(void);
#endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */
```

（2）实现系统调用功能（在 lab8 基础上修改）

vim kernel/sys.c

```

SYSCALL_DEFINE0(schello)
{
    struct task_struct *p;
    char state_char;

    printk("Hello new system call schello! hello LZH2311451\n\n");

    printk("%-20s %-6s %-6s %-6s\n", "Name", "Pid", "PPid", "Stat");
    for (p = &init_task; (p = next_task(p)) != &init_task; ) {
        state_char = task_state_to_char(p);
        printk("%-20s %-6d %-6d %-6c\n",
            p->comm, p->pid, p->real_parent->pid, state_char);
    }
    return 0;
}

```

(3) 注册系统调用 (lab8 已完成, 跳过)

`vim arch/x86/entry/syscalls/syscall_64.tbl`

466	common	removexattrat	sys_removexattrat
467	common	open_tree_attr	sys_open_tree_attr
468	common	schello	sys_schello

(4) 编译内核

增量编译 `make -j8`

(5) 安装内核模块

`sudo make modules_install`

(6) 安装内核

`sudo make install`

(7) 重启系统

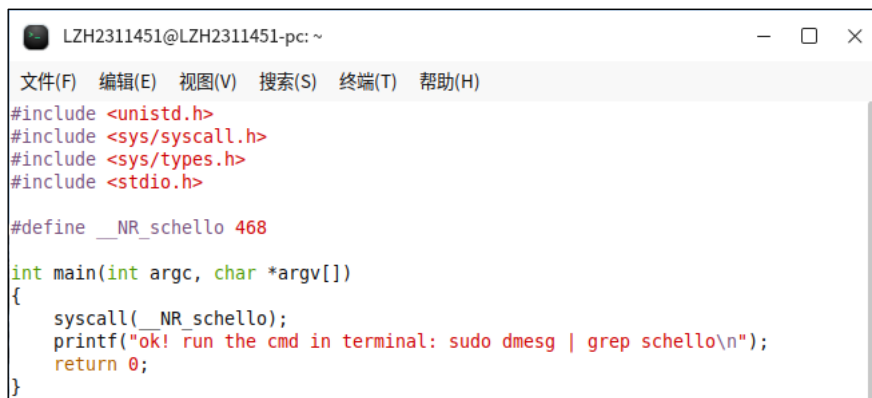
`sudo reboot`

4、编写用户态测试程序

lab8 已完成, 无需修改

`cd ~`

`vim test_schello.c`



```

LZH2311451@LZH2311451-pc: ~
文件(F) 编辑(E) 视图(V) 搜索(S) 终端(T) 帮助(H)
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <stdio.h>

#define __NR_schello 468

int main(int argc, char *argv[])
{
    syscall(__NR_schello);
    printf("ok! run the cmd in terminal: sudo dmesg | grep schello\n");
    return 0;
}

```

5、运行程序并验证结果

直接运行 lab8 编译过的可执行文件

查看内核日志时修改参数：grep -A 50 表示显示匹配行后 50 行

```
LZH2311451@LZH2311451-pc:~$ ./test_schello
ok! run the cmd in terminal: sudo dmesg | grep schello
LZH2311451@LZH2311451-pc:~$ sudo dmesg | grep -A 50 'schello!'
[ 2594.585397] Hello new system call schello! hello LZH2311451

[ 2594.585526] Name                Pid    PPid   Stat
[ 2594.585537] systemd                  1      0      S
[ 2594.585542] kthreadd                  2      0      S
[ 2594.585545] pool_workqueue              3      2      S
[ 2594.585549] kworker/R-rcu_g             4      2      I
[ 2594.585551] kworker/R-sync              5      2      I
[ 2594.585554] kworker/R-kvfre            6      2      I
[ 2594.585556] kworker/R-slub              7      2      I
[ 2594.585558] kworker/R-netns           8      2      I
[ 2594.585561] kworker/u512:0            12     2      I
[ 2594.585564] kworker/R-mm_pe           13     2      I
[ 2594.585567] ksoftirqd/0             14     2      S
[ 2594.585569] rcu_preempt               15     2      I
[ 2594.585572] rcu_exp_par_gp          16     2      S
[ 2594.585575] rcu_exp_gp_kthr          17     2      S
[ 2594.585577] migration/0              18     2      S
[ 2594.585580] idle_inject/0           19     2      S
[ 2594.585583] cpuhp/0                  20     2      S
[ 2594.585586] cpuhp/1                  21     2      S
[ 2594.585693] idle_inject/1            22     2      S
[ 2594.585698] migration/1            23     2      S
[ 2594.585701] ksoftirqd/1            24     2      S
[ 2594.585704] cpuhp/2                  27     2      S
[ 2594.585706] idle_inject/2            28     2      S
[ 2594.585709] migration/2            29     2      S
[ 2594.585801] ksoftirqd/2            30     2      S
[ 2594.585805] kworker/2:0H          32     2      I
[ 2594.585808] cpuhp/3                  33     2      S
[ 2594.585810] idle_inject/3            34     2      S
[ 2594.585812] migration/3            35     2      S
[ 2594.585815] ksoftirqd/3            36     2      S
[ 2594.585818] cpuhp/4                  39     2      S
[ 2594.585820] idle_inject/4            40     2      S
[ 2594.585822] migration/4            41     2      S
[ 2594.585825] ksoftirqd/4            42     2      S
[ 2594.585828] kworker/4:0H          44     2      I
[ 2594.585830] cpuhp/5                  45     2      S
[ 2594.585833] idle_inject/5            46     2      S
[ 2594.585835] migration/5            47     2      S
[ 2594.585838] ksoftirqd/5            48     2      S
[ 2594.585840] kworker/5:0H          50     2      I
[ 2594.585843] cpuhp/6                  51     2      S
[ 2594.585845] idle_inject/6            52     2      S
[ 2594.585847] migration/6            53     2      S
[ 2594.585850] ksoftirqd/6            54     2      S
[ 2594.585852] kworker/6:0H          56     2      I
[ 2594.585855] cpuhp/7                  57     2      S
[ 2594.585857] idle_inject/7            58     2      S
[ 2594.585860] migration/7            59     2      S
```

可以看到内核日志中输出了 hello 信息，并正确输出了进程信息。

三、结论

本次实验成功对 schello 自定义系统调用进行了修改，实现了进程信息遍历与打印功能。再次经历了修改内核源码、注册系统调用号、编译内核等步骤，体验了从系统调用开发到测试验证的全流程，加深了对 Linux 内核系统调用的底层实现机制与调用流程的理解。测试结果表明，schello 系统调用可正确输出系统中所有进程的名称、PID、父 PID 及状态信息，实验成功。